

Løsningsforslag TDT4110 H24 0900 og INFT1010 H24

Emne: TDT4110 / INFT1010 Informasjonsteknologi grunnkurs

Eksamensdato: mandag 25. november 2024 (0900-1300)

1 Eksakt 3%

Hvilke av disse tallene kan representeres eksakt som enten heltall eller flyttall i Python?

Hvis man skriver uttrykket i venstre kolonne, så velg "Heltall" hvis tallet lagres som heltall, "Eksakt flyttall" hvis det lagres som flyttall med den eksakte verdien til uttrykket, og "Ikke eksakt" ellers.

For hvert tall, sett kryss i rett kolonne. Ingen minuspoeng for feil svar, så du BØR svare noe også der du er usikker.

	Eksakt flyttall	Ikke eksakt	Heltall
$1+10^{**}20$	☐	☐	☒
$1 + 1/4$	☒	☐	☐
$11^{**}30 / 1$	☐	☒	☐
42	☐	☐	☒
1.1	☐	☒	☐
$1 + 1/2^{**}20$	☒	☐	☐

Maks poeng: 3

2 Regnerekkefølge 2%

Skriv svaret i boksen. Skriv svaret så enkelt som mulig
Det er ingen minuspoeng for feil svar.

a) $(7 + 3) * 5 - 2^{**}2$ (46)

b) $4 < 3 \text{ and } 2 > 2$ (False)

c) $\text{False or } 1 + 1 == 6 \% 4$ (True)

d) $5 + 2 == 7 \text{ and } 1$ (1)

Maks poeng: 2

3 Kodesnutt 2%

Hva blir resultatet av følgende kodesnutt?

```
1  flag = ""
2  a = 0
3  i = 1
4  while(a < 3):
5      j = 1
6      if flag:
7          i = j * i + 5
8      else:
9          i = j * i + 1
10     a = a + 1
11     print(i)
```

Velg ett alternativ:

- ☒ 4
- ☐ 16
- ☐ 11
- ☐ 12



Maks poeng: 2

4 Frukt 2%

Hva blir resultatet av følgende kodesnutt?

```
1 frukt = ["eple", "banan", "pære"]
2 frukt[1:2] = ["mango", "kirsebær"]
3 print(frukt)
```

Velg ett alternativ:

- ☐ ['mango', 'kirsebær', 'pære']
- ☒ ['eple', 'mango', 'kirsebær', 'pære']
- ☐ ['eple', 'mango', 'kirsebær']
- ☐ ['eple', 'banan', 'pære']



Maks poeng: 2

5 Python programmering 2%

Hva blir resultatet av følgende kodesnutt?

```
1  ord = "Python programmering"
2  n = len(ord)
3  ord1 = ord.upper()
4  ord2 = ord.lower()
5  nyttord = ""
6  for i in range(n):
7      if i % 2 == 0:
8          nyttord += ord2[i]
9      else:
10         nyttord += ord1[i]
11  print(nyttord)
```

Velg ett alternativ:

- ☒ pYtHoN PrOgRaMmErInG
- ☐ PyThOn pRoGrAmMeRiNg
- ☐ PYTHON PROGRAMMERING
- ☐ python programming



Maks poeng: 2

6 Matrise 2%

Hvilket av følgende alternativer produserer resultatet 6 når A er definert som

```
1  A = [[13, -9, -15],
2      [-5, 2, 6],
3      [8, 0, 12]]
4
```

Velg ett alternativ:

- ☐ A[1][2]
- ☐ A[2][3]
- ☐ A[3][2]
- ☐ A[2][1]



Maks poeng: 2

7 Synlighet 2%

Hva blir resultatet av følgende kodesnutt?

```
1  verdi = "herlig"
2
3  def minfunksjon():
4      global verdi
5      verdi = "fantastisk"
6
7  minfunksjon()
8  print("Python er " + verdi)
```

Velg ett alternativ:

- ☐ Python er fantastisk
- ☐ error
- ☐ undefined
- ☐ Python er herlig



Maks poeng: 2

8 Muterbarhet 2%

Hvilken av følgende datatyper er muterbar i Python?

Velg ett alternativ:

- ☐ list
- ☐ tuple
- ☐ int
- ☐ float



Maks poeng: 2

9 Nøstet ordbok 2%

Hva blir korrekt syntaks for å skrive ut navnet "Håkon" gitt at du har følgende datastruktur?

```
1 a = {'navn' : 'Kristin', 'alder' : '20'}
2 b = {'navn' : 'Håkon', 'alder' : '23'}
3 kunder = {'c1' : a, 'c2' : b}
```

Velg ett alternativ:

- ☐ print(kunder.c2.b['navn'])
- ☐ print(kunder.c2.navn)
- ☐ print(kunder['c2']['navn'])
- ☐ print(kunder['c2'].navn)



Maks poeng: 2

10 Import 2%

Hva skriver du om du bare ønsker å importere funksjonen sqrt fra math-modulen?

Velg ett alternativ:

- ☐ from math import sqrt
- ☐ from sqrt import math
- ☐ import sqrt from math
- ☐ import math.sqrt



Maks poeng: 2

11 Funksjonen regnut() 2%

```
def regnut(a):  
    a = a + 2  
    a = a * 2  
    return a  
print(regnut(2))
```

Hva er resultatet av følgende program?

Velg ett alternativ:

- ☐ 8
- ☐ Runtime Error
- ☐ 4
- ☐ Indentation Error



Maks poeng: 2

12 Tallsekvens 2%

Nedenfor har vi en for-løkke som skriver ut en sekvens med tall.
Hvilke tall vil bli skrevet ut av løkkene?

```
1  for i in range(1,7,3):  
2      print(i)
```

OBS! Gale svar på denne oppgaven gir minuspoeng.

Velg ett eller flere alternativer

☐ 8

☐ 7

☐ 2

☐ 4



☐ 5

☐ 3

☐ 6

☐ 1



☐ 0

Maks poeng: 2

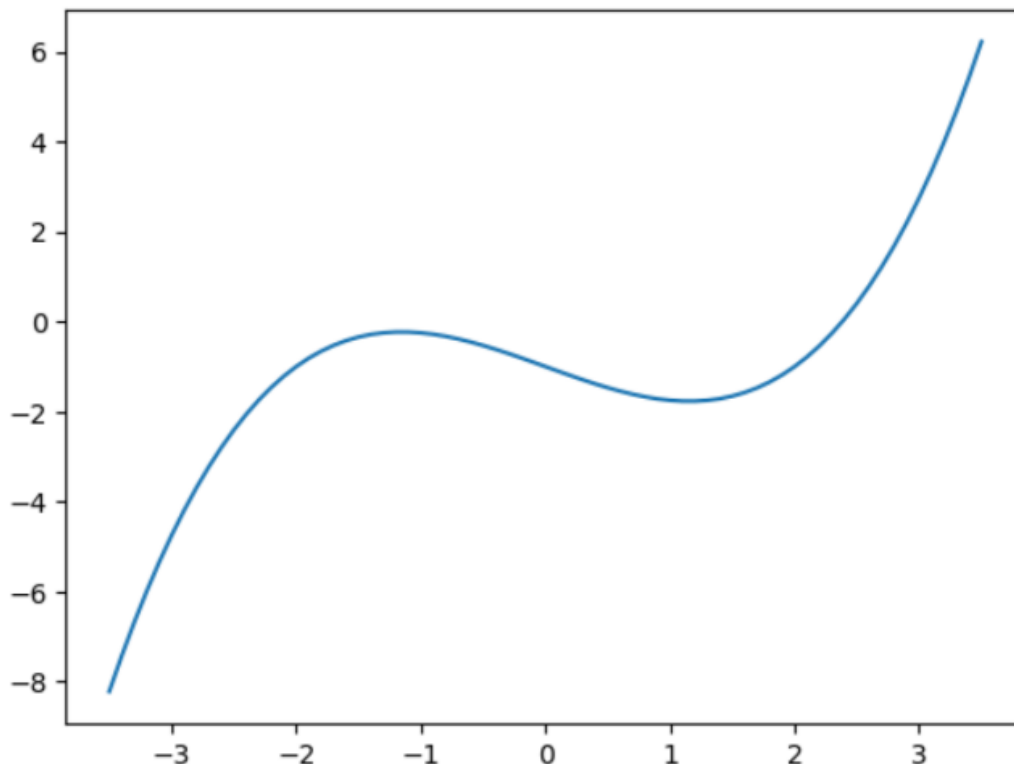
13 Plotte graf 4%

Gitt følgende kode:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def f(x):
    return 0.25*x**3 - x - 1
```

Vi ønsker nå å plotte funksjonen f over intervallet $-3.5 \dots 3.5$, som vist på figuren, dvs. startpunktet på den blå kurven er $x = -3.5$, $y = -8.22$ og sluttpunktet er $x = 3.5$, $y = 6.22$.



Trekk kodelinjer til riktig plass så vi får det ønskede plottet. Kun 4 av kodelinjene skal brukes.

Hjelp

`plt.show()`

`x = np.arange(-3.5, 3.6, 0.1)`

`x = np.arange(-3.5, 3.5, 0.1)`

`np.show()`

`plt.subplots()`

`ax.plot(x, f(x))`

fig, ax = `plt.subplots()` ✓

`x = np.arange(-3.5, 3.6, 0)` ✓

`ax.plot(x, f(x))` ✓

`plt.show()` ✓

Maks poeng: 4

14 Iterere 2d array 7%

Funksjonen **check_array(arr, c, d)** skal få inn tre parametre arr, c, d, hvor arr er et numpy array med tall, mens c og d er heltall. Funksjonen skal returnere **True** hvis inntil **c** av radene i arrayet arr inneholder minst ett partall som er $< d$, og **False** hvis mer enn c rader har partall $< d$. Eksempel:

Hvis arrayet arr er

```
[ [5 -1 1]
  [5 6 7]
  [2 9 1]]
```

så skal

- `check_array(arr, 1, 4)` gi **True** fordi bare 1 rad (nederste) har et partall < 4 (nemlig 2).
- `check_array(arr, 1, 7)` gi **False** fordi det er mer enn én rad (de to nederste) som har partall < 7 .

Velg riktig alternativ i hullene slik at funksjonen virker som den skal:

```
def check_array(arr, c, d):
    count = 0
```

Velg alternativ (for row in range(count);, for row in range(len(arr));, for row in arr:)
for number in row:

Velg alternativ (if number // 2 == 0; if number < d or number % 2 == 0; if number < d and number % 2 == 0:)

Velg alternativ (return True, return False, count += 1)

Velg alternativ (return True, return False, break)

Velg alternativ (if count > c; if count == 0; if count < c:)

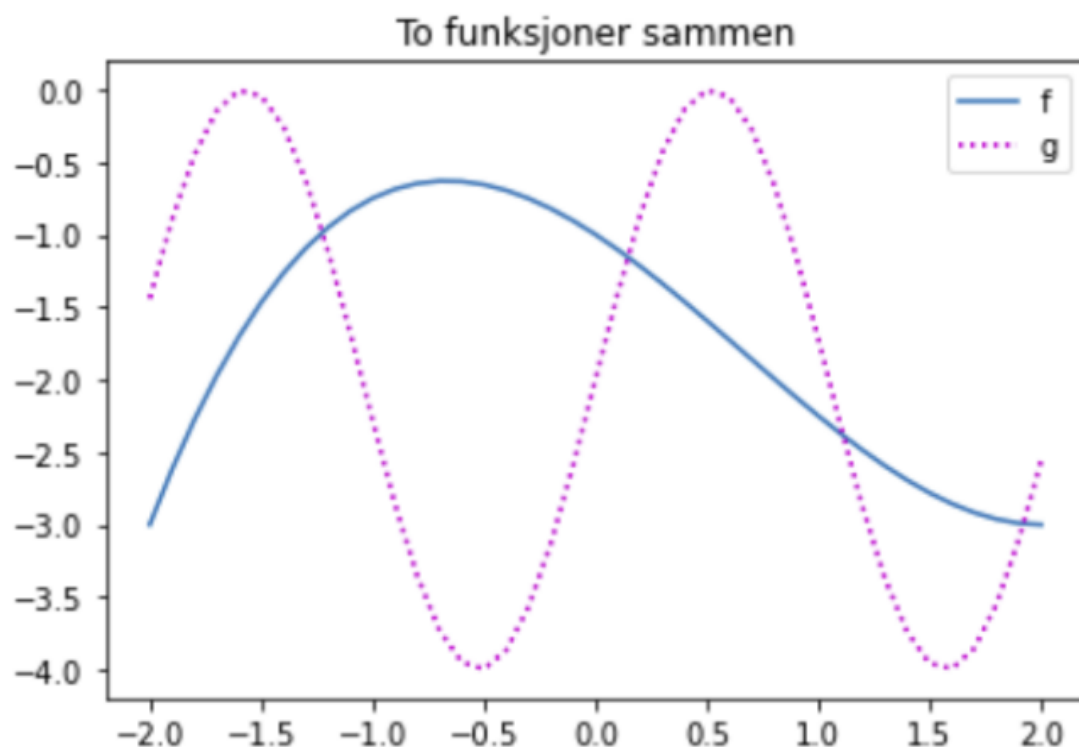
Velg alternativ (return False, return True, break)

Velg alternativ (return False, return True, return ok)

Maks poeng: 7

15 Plotte to funksjoner 6%

Koden under skal plotte to funksjoner i samme bilde, som vist her. Merk spesielt den lille boksen i hjørnet som viser hvilken graf som tilhører hvilken funksjon. For å skille kurvene er ulike farger brukt, samt at den ene har heltrukken linje og den andre prikkete. Det reguleres med et tredje argument til plot-funksjonen. I et kall som `ax.plot(x, y, "m")` sier tegnet ":" at vi skal ha prikkete linje (mens f.eks. "-" ville gi streket linje, og "o" store rundinger), og "m" angir farge magenta (mens f.eks. "g" - grønn, "r" - rød, "b" - blå, "k" - svart). Merk deg også hvorvidt kurvene går over samme x-intervall eller ikke, samt om de er glatte eller hakkete, som indikerer om det er mange eller få verdier i x-vektoren som ble plottet.



Velg rett alternativ i hvert tomrom i koden under, slik at den vil resultere i eksakt samme figur som vist.

```

Velg alternativ (from numpy import *, import numpy as np, import numpy)
import matplotlib.pyplot as plt

def f(x):
    return 0.25*x**3 - 0.5*x**2 - x - 1

def g(x):
    return 2 * np.sin(3*x) - 2

fig, ax = plt.subplots()

x = np. Velg alternativ (linspace(-2.0, 2.0, 9), arange(-2.0, 2.1, 0.1), arange(-2.0, 2.1, 0.5))

ax. Velg alternativ (plot(x, f(x)), plot(f(x), x), plot(x, f(x), ":m"))

Velg alternativ (# ingen kode trengs her, v = np.arange(-2.0, 3.0, 0.1), v = np.arange(-3-0, 2.0, 0.001))

ax. Velg alternativ (plot(x, g(x), ":m"), plot(x, g(x)), plot(v, g(v), ":m"))

ax.legend( Velg alternativ (["f(x)", "g(x)"], ["f", "g"], ["f", "g"]))
plt.title("To funksjoner sammen")
plt.show()

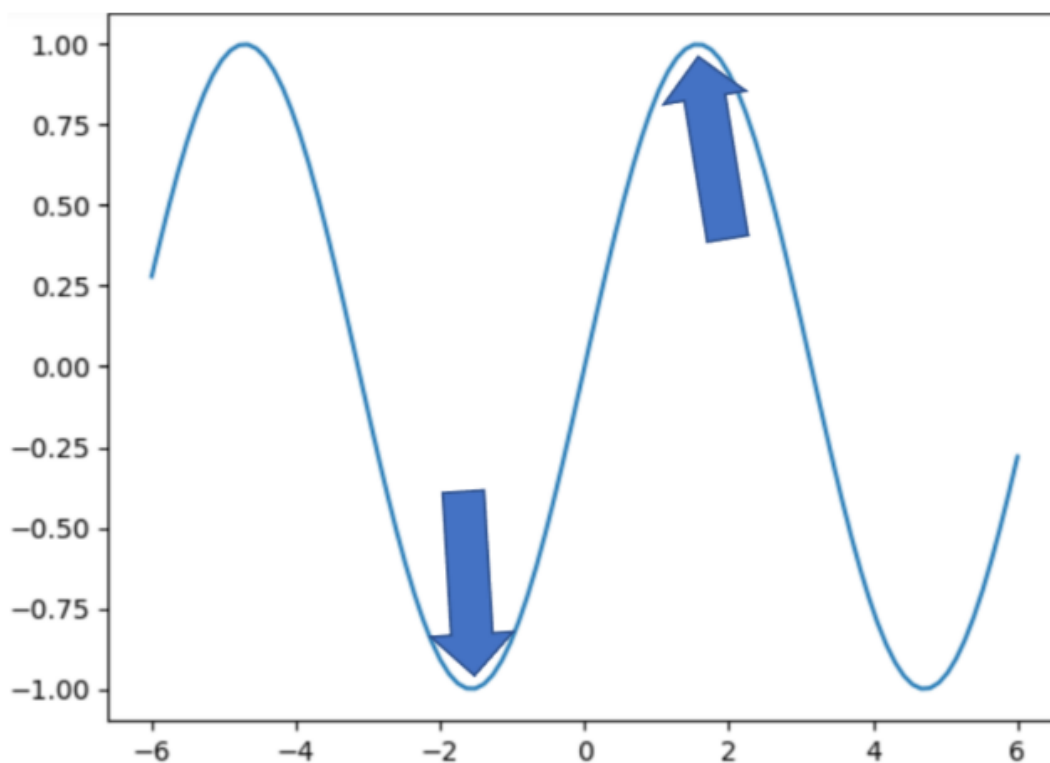
```

Maks poeng: 6

16 Funksjoner som parametre 8%

Vi ønsker en funksjon **longest_increase(f, x_values)** som får inn som parametre en funksjon **f** og et array med flyttall **x_values**. Funksjonen skal returnere et tuppel som angir første og siste x-verdi i det lengste intervallet innenfor **x_values** hvor **f** er strengt stigende. Som et eksempel, anta at funksjonen som gis inn for parameter **f** er **np.sin(x)** som vist i plottet under, og at **x_values** er **np.arange(-6.0, 6.0, 0.01)**. Funksjonen skal da finne intervallet fra $x = -1.57$ til $x = 1.57$ siden dette er det lengste strekket hvor funksjonsverdien stiger over arrayet **x_values**, fra bunnpunktet til toppunktet vist med blå piler (bunnpunktet for sinuskurve er på cirka $x = -1.571$ og toppunktet på $x = 1.571$, men stegverdien 0.01 på arange begrenser hvor nær vi kan komme de eksakte verdiene). Altså, **longest_increase(np.sin(x), np.arange(-6.0,6.0,0.01))** skal returnere **(-1.57, 1.57)**.

Hvis det ikke fins noen strekning hvor **f** er stigende i det gitte intervallet, eller **x_values** er tom eller bare inneholder ett element, skal funksjonen **longest_increase** returnere et **tomt tuppel**.



Velg riktig i feltene under slik at funksjonen virker som den skal.

```
import numpy as np
```

```
def longest_increase(f, x_values):  
    result = ()  
    current_start = None  
    for i in range(1, len(x_values)):  
        if f(x_values[i]) > f(x_values[i-1]):
```

Velg alternativ (if current_start is None:, if f(x_values[i]) > f(x_values[i-1]):, if result == ():)

Velg alternativ (current_start = x_values[i-1], result[0] = current_start, current_start = result[0])

Velg alternativ (elif current_start is None:, elif result = ():, elif i == len(x_values) - 1:)

Velg alternativ (if result == () and not (x_values[i] - current_start > result[1] - result[0]):, if x_values[i] - current_start > result[1] - result[0]:, if result == () or (x_values[i] - current_start > result[1] - result[0]):)

Velg alternativ (result = (current_start, x_values[i]), result = (result[0], x_values[i] - current_start), result = (f(x_values[i]), f(x_values[i-1])))
else:

Velg alternativ (if result = ():, if current_start is None:, if current_start is not None:)

Velg alternativ (if result == ():, if result == () or (x_values[i - 1] - current_start > result[1] - result[0]):, if x_values[i - 1] - current_start > result[1] - result[0]:)
result = (current_start, x_values[i - 1])

Velg alternativ (current_start = f(x_values[i]), current_start = x_values[i], current_start = None)
return result

Maks poeng: 8

1.17 Dyretelling 4%

Nå skal du løse funksjonen `les_registreringer()`, av dyreliv i Norge. Først litt ekstra informasjon, så kommer kravene bedre beskrevet nær der koden skal spesifiseres:

Innholdet i eksempelfilen 'dyr.txt'.

```
elg 10
ku 13 16 2
elg 5
svin 4 12
elg 13 1
marsvin 3
kråke 2
maur 103463
```

Eksempel på bruk med denne filen:

```
>>> print(les_registreringer('dyr.txt', ['rev', 'elg', 'bjorn']))
{'elg': [10, 5, 13, 1]}
>>> print(les_registreringer('dyr.txt', ['maur', 'elg', 'bjorn']))
{'elg': [10, 5, 13, 1], 'maur': [103463]}
```

Slep og slipp elementene under for å kode funksjonen `les_registreringer(...)`. Bruk tekstfilen og eksempelet over til å hjelpe deg.

- Funksjonen har to parametre:
 - `filnavn`: filnavnet som skal leses. Du kan forutsette at filen ligger i samme mappe.
 - `dyr`: En liste med strenger som beskriver hvilke dyr som skal registreres.
- En registrering består av et dyrenavn, et mellomrom, og så ett eller flere tall separert med mellomrom. (elg 12 eller ku 12 14)
- Funksjonen skal ignorere eventuelle tomme linjer, eller om registreringene omhandler dyr som ikke er omtalt i listen 'dyr'.
- Funksjonen skal returnere en dictionary med følgende struktur:
 - Nøklerne skal være dyrene som vi ønsket å lese registreringene til.
 - Verdiene skal være en liste over de ulike registreringene til det spesifikke dyret, i rekkefølgen det ble lest inn.

parts[0]	result[dyrenavn]	parts.keys()	extend	line
line = line.strip()	line.isEmpty():	read()	parts[:1]	readline()
add	parts	line.split()	append	parts[1:]

```
def les_registreringer(filnavn, dyr):
    result = {}
    with open(filnavn, 'r') as file:
        for line in file:
            line = line.strip() ✓
            if line : ✓
                parts = line.split() ✓
                dyrenavn = parts[0] ✓
                registreringer = []
                for x in parts[1:] : ✓
                    registreringer.append(int(x)) ✓
                if dyrenavn in dyr:
                    if dyrenavn not in result:
                        result[dyrenavn] = [] ✓
                    result[dyrenavn].extend(registreringer) ✓
    return result
```

18 De fleste skal bli de første 6%

I denne oppgaven skal du bruke resultatet av funksjonskallet til forrige oppgave. Slep og slipp elementene for å løse funksjonen `sort_by_registrations` i henhold til spesifikasjonen under:

- Funksjonen har én parameter, `registreringer` (dictionary med nøkkel dyreart (string), verdi en liste med heltallsregistreringer) som for eksempel `{'elg': [10, 5, 13, 1], 'maur': [103463]}`
- Funksjonen skal skrive ut summen av hver enkelt dyrearts registreringer.
- Rekkefølgen på utskriften avhenger av antall dyr registrert, høyest antall registreringer først.
- Utskriften skal følge følgende mal (utelat <>):
 - <dyreart>: <totalt antall registreringer> (<hvor mange færre enn den over, på øverste>)

Eksempel merk at det fløy inn en ekstra undulat:

```
>>> r = {'elg': [10, 5, 13, 1], 'maur': [103463], 'undulat': [1]}
>>> sort_by_registrations(r)
maur: 103463 (0)
elg: 29 (103434)
undulat: 1 (28)
```

```
def sort_by_registrations(registreringer):
```

```
     (stuffed, 
    (registreringer.keys(), [], set(), {})
```

```
    for  (variable,, Alternativ 4, variable, 
    (
```

```
        important[variable] =  (len(something), something[-1], 
something)
```

```
        stuffed = []
```

```
         (Alternativ 4, for, if, while) variable in  (Alternativ 4, 
important[1:], important.items()):
```

```
             (stuffed.extend(
important[variable]))
```

```
        for i in range(len(stuffed)):
            for j in range(i + 1, len(stuffed)):
                if stuffed[i][1] < stuffed[j][1]:
                    stuffed[i], stuffed[j] = stuffed[j], stuffed[i]
```

```
        also_important = None
```

```
        for variable, total in  (important, registreringer, stuffed.items(), 

```
 if also_important is (stuffed[1:], stuffed, stuffed.items(),
```


```

```
                print(f"{variable}: {total} ({ (len(variable), len(also_important[variable]),"/>
also_important, variable) - total}))")
```

```
             (also_important +, variable, 
```

19 Les kamper 10%

I denne oppgaven skal vi lese resultater fra kamper i fotball eller liknende idrett.

Hver linje en tekstfil har resultatet fra en kamp skrevet på formen

hjemmelag;bortelag:resultat

Resultat angir antall mål som henholdsvis hjemmelaget og bortelaget har scoret. Se eksempel lenger ned.

Skriv funksjonen **les_kamper(filnavn, logger)** som leser alle radene i filen og lagrer resultatene i en ordbok, som returneres fra funksjonen.

Elementene i ordboka skal ha hjemmelag som nøkkel. Verdien er en ny ordbok. Her er nøklene bortelagene som hjemmelaget har spilt mot. Verdiene til denne er en liste med resultatene til alle kampene mellom de to lagene, med hjemmelaget først. Kamper hvor hjemmelaget spilte borte, vil da ligge under bortelaget.

Eksempel: Gitt en fil med følgende innhold:

```
Lag1;Lag2:2-1
Lag2;Lag3:0-0
Lag1;Lag3:1-2
Lag3;Lag1:0-1
Lag2;Lag1:1-1
Lag2;Lag1:4-2
```

så skal funksjonen returnere en dictionary som ser slik ut:

```
{'Lag1': {'Lag2': ['2-1'], 'Lag3': ['1-2']}, 'Lag2': {'Lag3': ['0-0'], 'Lag1': ['1-1', '4-2']}, 'Lag3': {'Lag1': ['0-1']}}
```

Resultatene er lagret som en streng, hvor det første tallet er antall mål hjemmelaget har scoret, og det andre tallet er antall mål bortelaget har scoret.

Input-parameteren logger er en funksjon, som tar inn en streng, og returnerer ingenting (None). Den skal brukes til å logge feil i filen. Hvis den er None, så skal vi skrive til terminal med print.

Hvis det oppstår en feil ved lesning i en linje, så skal feilmeldingen logges, med kall til logger-funksjonen eller skrives til terminalen.

Hvis det oppstår en feil ved åpning av filen, så skal dette logges, og det returneres en tom ordbok. I motsatt fall logges det at filen er lest når alle linjene er lest.

OPPGAVE: Trekk kodelinjer til rett sted slik at de fire funksjonene virker som de skal. Ingen minuspoeng for feil svar, så du BØR svare noe også der du er usikker. Alle kodelinjene skal brukes.

else:	except Exception as e:
return hjkamper	hjemmelag, bortelag = lagene.split(";")
if hjemmelag not in hjkamper:	hjkamper[hjemmelag][bortelag].append(resultat)
try :	logger = print
hjkamper[hjemmelag] = {}	{ }
lagene, resultat = linje.strip().split(":")	hjkamper[hjemmelag][bortelag] = []
logger(f"Ferdig å lese filen {filnavn}")	logger(e)
except IOError as e:	for linje in f:
if bortelag not in hjkamper[hjemmelag]:	with open(filnavn, "r", encoding="utf-8") as f:

def les_kamper(filnavn, logger):

if not logger:

logger = print

hjkamper =

{ }

try :

with open(filnavn, "r", encoding="utf-8") as f:

for linje in f:

try:

lagene, resultat = linje.strip().split(":")

hjemmelag, bortelag = lagene.split(";")

if hjemmelag not in hjkamper:

hjkamper[hjemmelag] = {}

if bortelag not in hjkamper[hjemmelag]:

hjkamper[hjemmelag][bortelag] = []

hjkamper[hjemmelag][bortelag].append(resultat)

except Exception as e:

logger(e)

except IOError as e:

logger(e)

else:

logger(f"Ferdig å lese filen {filnavn}")

return hjkamper

20 Skihopp 6%

I skihopp får man poeng basert på lengde og stil,

- **Lengdepoeng:** Hoppere får 60 poeng for å nå K-punktet (standard lengde i bakken). For hvert meter over K-punkt, legges det til 1.8 poeng, og for hver meter under trekkes det 1.8 poeng.
- **Stilpoeng:** Fem dommere gir stilpoeng på en skala fra 0 til 20. Det høyeste og laveste poenget fjernes, og de tre midterste poengene legges sammen.

Lag funksjonen **beregnPoeng()** som returnerer antall poeng for et gitt skihopp. Inputparametere til funksjonen er **lengde**, **stilpoeng** og **kpunkt**.

Stilpoengene kommer på formen **stilpoeng = [p1,p2,p3,p4,p5]** der elementene i listen er stilpoeng fra hver dommer.

Løsningsforslag

```
1  def beregnPoeng(lengde, stilpoeng, kpunkt):
2
3      # Lengdepoeng
4      poeng = 60 + (lengde - kpunkt) * 1.8
5
6      # Stilpoeng
7      stilpoeng.sort()
8      poeng += sum(stilpoeng[1:4])
9
10     return poeng
```

21 Generere passord 6%

Vi skal lage en enkel funksjon som genererer passord.

Følgende kriterier må være oppfylt:

- Passordet skal være minst 12 tegn.
- Passordet må inneholde både store og små bokstaver.
- Passordet må inneholde tall.
- Passordet må inneholde spesialtegn.

Lag kode for funksjonen **lagPassord(lengde)** som returnerer en tekststreng med et nytt tilfeldig passord av en bestemt lengde i henhold til reglene ovenfor. Om lengden som gis som input er under 12 skal den settes til 12 som er minimum.

Løsningsforslag

```
import random
import string

def lagPassord(lengde):

    if lengde < 12:
        lengde = 12

    symboler = (string.ascii_lowercase,
                string.ascii_uppercase,
                string.digits,
                string.punctuation)

    # Alternativt uten å bruke string-modulen:
    # symboler = ("abcdefghijklmnopqrstuvwxyzæøå",
    #             "ABCDEFGHIJKLMNOPQRSTUVWXYZÆØÅ",
    #             "0123456789",
    #             "!\"#$%&/'()=?`^~*'-_.:;,"")

    passord = []

    # Sikre minst en av hver type
    for symbol_gruppe in symboler:
        passord.append(random.choice(symbol_gruppe))

    # Resten er tilfeldig valgt
    alle_symboler = "".join(symboler)
    passord += random.choices(alle_symboler, k = lengde - len(passord))

    # Lag tilfeldig rekkefølge, slik at de fire første også blir tilfeldig plassert
    random.shuffle(passord)

    # random.choices() returnerer liste, så vi må lage streng som returneres
    return "".join(passord)
```

22 Kjennemerke for bil 6%

Lag en funksjon **nyttRegNr()** som genererer og returnerer et tilfeldig kjennemerke for bil. Type kjøretøy gis inn som parameter i funksjonen og kan være enten gass, elektrisk, hydrogen eller fossil.

Kjennemerket, også kalt bilnummeret, skal bestå av to bokstaver etterfulgt et tall bestående av fem sifre. Bokstavene I, M, O, Q, Æ, Ø og Å brukes ikke på grunn av likheten med andre bokstaver eller sifre. 0 kan ikke være det første av de 5 sifrene i tallet.

Noen bokstavkoder er reservert. Dersom kjøretøyet drives med gass skal funksjonen sette bokstavkoden til GA, hydrogen til HY og elektrisk til EJ. Disse er ikke tilgjengelig om bitypen er fossil. Bokstavkoden CD er reservert ambassadebiler og skal ikke kunne genereres.

Løsningsforslag

```
import random

def nyttRegNr(biltype):
    bokstaver = ""
    if biltype == "gass":
        bokstaver = "GA"
    elif biltype == "elektrisk":
        bokstaver = "EJ"
    elif biltype == "hydrogen":
        bokstaver = "HY"
    elif biltype == "fossil":
        while True:
            bokstaver = "".join(random.choices("ABCDEFGHJKLNPRSTUVWXYZ", k=2))
            if bokstaver not in ["EJ", "HY", "GA", "CD"]:
                break
    else:
        print("Ukjent biltype")
        return None

    return bokstaver + str(random.randint(10000, 99999))
```

23 Cæsarchiffer 6%

Innen kryptografi er cæsarchiffer, også kjent som Caesars chiffer, skiftchiffer, Cæsars kode og cæsarskift, en av de enkleste og mest kjente krypteringsteknikker. Det er en type substitusjonschiffer der hver bokstav i klarteksten erstattes med en annen bokstav et gitt antall steg lenger ut i alfabetet. Med et skift på tre steg erstattes D med A, E med B, F med C og så videre. Metoden har navn etter Julius Cæsar som benyttet denne koden til å kommunisere med sine generaler.

Lag funksjonen **krypter(tekst,skift)** som krypterer en tekst med cæsarchiffer. Inputvariabelen skift forteller hvor mange steg en bokstav forflytter seg i alfabetet.

Vår funksjon er noe forenklet og skal kun ta hensyn til bokstaver i det norske alfabetet. Funksjonen skal returnere den krypterte teksten med store bokstaver, og alle mellomrom skal være fjernet.

Eksempelvis vil **krypter("Kryptering er gøy",1)** returnere den krypterte teksten **LSZQUFSJOHFSHÅZ**

Løsningsforslag

```
1  def krypter(tekst,skift):
2
3      bokstaver = 'abcdefghijklmnopqrstuvwxyzæøå'
4
5      oversatt = ""
6
7      for x in tekst:
8          if x.lower() in bokstaver:
9              num = bokstaver.find(x.lower())
10             num = num + skift
11
12             if num < 0:
13                 num = num + len(bokstaver)
14
15             if num >= len(bokstaver):
16                 num = num - len(bokstaver)
17
18             oversatt += bokstaver[num]
19         else:
20             oversatt += x
21     return oversatt.upper().replace(" ", "")
```

Alternativ løsning:

```
def krypter(tekst, skift):
    tegn = "abcdefghijklmnopqrstuvwxyzæøå"
    kryptert = ""
    for t in tekst.lower():
        if t in tegn:
            kryptert += tegn[(tegn.find(t) + skift) % len(tegn)]
    return kryptert.upper()
```

24 Sjakkbrett 6%

Tenk deg et sjakkbrett med 8x8 ruter kodet som en 2-dimensjonal matrise.

De forskjellige brikkene er representert ved forskjellige bokstaver. For eksempel er kongene kodet med bokstaven "K". Vi bryr oss i denne oppgaven ikke om andre brikker, ei heller skiller vi mellom hvit eller sort konge.

Figuren under viser en visualisering av denne matrisen med 2 konger plassert i hver sin rute.

	A	B	C	D	E	F	G	H
8								
7								
6					K			
5								
4								
3								
2		K						
1								

Skriv en funksjon **opprettSjakkbrett()** som oppretter en 2-dimensjonal matrise og legger inn to konger på to tilfeldige plasser i matrisen. Figuren over er kun et eksempel og vi vet ikke i hvilke av rutene som kongene havner i. La funksjonen returnere matrisen.

Skriv en ny funksjon **finnKonger()** som løper igjennom den samme matrisen og finner alle rutene med konger. Det vil si ruter i den 2-dimensjonale matrisen som inneholder bokstaven "K".

La denne funksjonen returnere koordinatene til rutene hvor det befinner seg konger på følgende format:

Konge E6

Konge B2

Løsningsforslag

```
1  import random
2
3  def opprettSjakkbrett():
4
5      global brett
6      brett = [["" for i in range(8)] for j in range(8)]
7
8      k1x = random.randint(0,7)
9      k1y = random.randint(0,7)
10     k2x = random.randint(0,7)
11     k2y = random.randint(0,7)
12
13     for x in range(8):
14         for y in range(8):
15             if (x == k1x and y == k1y) or (x == k2x and y == k2y):
16                 brett[x][y] = "K"
17
18     return brett
19
20 def finnKonger():
21     utdata = ""
22     bokstaver = "ABCDEFGH"
23
24     for x in range(8):
25         for y in range(8):
26             if brett[x][y] == "K":
27                 utdata += "Konge " + bokstaver[x] + str(y+1) + "\n"
28
29     return utdata
```